ПОДГОТОВКА К ЗАЩИТЕ ЛАБОРАТОРНЫХ РАБОТ № 2, 3 И 4

Вариант 13. М. П. Курбатов

Основа: методичка Аграновского, Боженко, Турнецкой; ваш отчет № 2 «Фильмы»; отчеты № 3 и № 4 «ГИБДД». Вопросы 1–5 взяты из самопроверки к ЛР № 2, вопросы 32–39 — из самопроверки к ЛР № 4. К ЛР № 3 отдельного списка самопроверки в подразделе 5.6 нет: вопросы составлены по требованиям и теории раздела 5. Остальные вопросы добавлены для защиты ваших конкретных решений. Ответы сформулированы для устного объяснения, а не как буквальные цитаты.

ЛАБОРАТОРНАЯ № 2. РЕАЛИЗАЦИЯ БАЗЫ «ФИЛЬМЫ»

1. Для чего нужен графический клиент? [Самопроверка методички]

MySQL Workbench позволяет проектировать модели, подключаться к серверу, выполнять SQL, просматривать данные, импортировать и экспортировать базы. Workbench — клиентская программа; данные хранит и запросы исполняет MySQL Server. Проектировать автономную модель можно без подключения к серверу.

2. Что означает перенос базы из модели на сервер? [Самопроверка методички]

По модели создается SQL-код, который выполняется на сервере. В результате на сервере появляются схема, таблицы, индексы и ограничения. Открытая диаграмма сама по себе не означает, что эти объекты уже созданы на сервере.

3. Что такое автогенерация программного кода по модели? [Самопроверка методички]

Workbench преобразует описание таблиц и связей в SQL-команды CREATE SCHEMA, CREATE TABLE и определения ключей. В вашей работе используется File → Export → Forward Engineer SQL CREATE Script. Сгенерированный код нужно проверить, сохранить и выполнить на сервере.

4. Что такое модель данных? [Самопроверка методички]

Это описание структур данных, ограничений и допустимых операций. В рамках лабораторной конкретная модель базы показывает, какие объекты хранятся, какие у них атрибуты и как они связаны.

5. Какие типы связей бывают? [Самопроверка методички]

Один к одному (1:1), один ко многим (1:N) и многие ко многим (M:N). В реляционной схеме M:N обычно реализуется промежуточной таблицей и двумя внешними ключами. В вашей базе films и clients имеют связи 1:N с issued_films.

6. Чем отличаются ваши три лабораторные?

В № 2 я прошел полный цикл на базе проката: модель, создание таблиц на сервере и заполнение. В № 3 спроектировал инфологическую и даталогическую модели «ГИБДД». В № 4 перенес модель «ГИБДД» на сервер, заполнил таблицы и создал дамп.

7. Какие таблицы есть в базе «Фильмы»?

films хранит код фильма, название, режиссера и год выхода; clients — код клиента, фамилию, имя и телефон; issued_films — фильм, клиента, дату выдачи и стоимость проката в день. Первые две таблицы являются справочниками, третья — журналом операций.

8. Чем отличаются первичный и внешний ключи?

PRIMARY KEY однозначно идентифицирует строку: значения уникальны и не могут быть NULL. FOREIGN KEY ссылается на ключ другой или той же таблицы и поддерживает ссылочную целостность. Сам по себе внешний ключ не делает значения своего столбца уникальными.

9. Какой ключ у issued_films и почему он составной?

Составной первичный ключ — (filmId, clientId, issueDate). Операцию выдачи определяет сочетание фильма, клиента и даты. Поля filmId и clientId одновременно являются внешними ключами.

10. Зачем issueDate входит в ключ? Есть ли ограничение такого решения?

Без даты один клиент не смог бы взять один и тот же фильм повторно. Дата допускает повторение в разные дни, но при типе DATE запрещает две выдачи того же фильма тому же клиенту в один день. Если такие операции нужны, можно добавить отдельный issueId; одной смены типа даты недостаточно без определения требований к точности времени.

11. Для чего AUTO_INCREMENT? Что будет после удаления строк?

Сервер автоматически выдает значение идентификатора при вставке без явного ID. Пропуски допустимы: автоинкремент не является счетчиком строк. DELETE обычно не сбрасывает счетчик; после удаления пяти строк следующий ID не обязан быть равен 1.

12. Почему для clientId в issued_films нужен дополнительный индекс?

Составной первичный индекс начинается с filmId, а clientId стоит вторым. Для внешнего ключа clientId нужен индекс, где этот столбец стоит первым; InnoDB при необходимости создает такой индекс автоматически. Для filmId подходит левый префикс составного PK.

13. Почему стоимость — DECIMAL, телефон — VARCHAR, а год — YEAR?

DECIMAL(10,2) хранит десятичную сумму с двумя знаками после запятой без погрешностей FLOAT. Телефон является строкой: в нем важны плюс и возможные начальные нули, арифметика над ним не нужна. YEAR предназначен для года, DATE — для календарной даты.

14. В каком порядке заполняются таблицы «Фильмы»?

Сначала films и clients, затем issued_films. Иначе внешний ключ не найдет фильм или клиента. Поскольку выдачи в отчете ссылаются на ID 1–5, перед их загрузкой нужно проверить реальные идентификаторы, а не предполагать, что AUTO_INCREMENT обязательно начал с 1.

15. Что дает IF NOT EXISTS? Гарантирует ли оно правильную структуру?

CREATE TABLE IF NOT EXISTS не создает таблицу повторно, если имя уже занято. Это не проверка совпадения структуры с моделью и не обновление таблицы. Для проверки

используются SHOW CREATE TABLE и DESCRIBE; повторный INSERT по-прежнему может вызвать дублирование ключа.

ЛАБОРАТОРНАЯ № 3. ПРОЕКТИРОВАНИЕ «ГИБДД»

16. Какова цель работы № 3?

Разработать инфологическую модель предметной области и даталогическую модель реляционной базы. Нужно определить сущности, атрибуты, ключи, связи, типы данных и привести модель к третьей нормальной форме. Результат — модели, а не обязательно заполненная база на сервере.

17. Что такое сущность, атрибут и экземпляр сущности?

Сущность — класс объектов предметной области, например «Автомобиль». Атрибут — его характеристика, например мощность. Экземпляр — конкретный автомобиль; в таблице ему соответствует одна строка.

18. Как читается нотация Чена?

Сущность изображается прямоугольником, атрибут — овалом, связь — ромбом. Ключевой атрибут подчеркивается, возле связей указываются мощности. Например, гражданин владеет автомобилями: одному гражданину может соответствовать несколько автомобилей.

19. Чем отличаются инфологическая и даталогическая модели?

Инфологическая описывает смысл предметной области: сущности, атрибуты и отношения без привязки к физическому хранению. Даталогическая переводит описание в выбранную модель данных: в моей работе это таблицы MySQL с PK, FK, типами и ограничениями.

20. Почему в «ГИБДД» пять таблиц?

citizens хранит граждан, cars — автомобили и текущих владельцев, accidents — обстоятельства ДТП, accident_cars — участие автомобилей в ДТП, sales — историю продаж. Промежуточная таблица нужна для M:N, а продажи выделены отдельно, потому что являются повторяющимися событиями.

21. Какие конкретно внешние ключи есть в «ГИБДД»?

Всего шесть: cars.ownerId → citizens.citizenId; accident_cars.accidentId → accidents.accidentId; accident_cars.carId → cars.carId; sales.carId → cars.carId; sales.sellerId → citizens.citizenId; sales.buyerId → citizens.citizenId. Две последние связи ведут к одной таблице, но обозначают разные роли гражданина.

22. Почему нельзя записать список автомобилей в одно поле accidents?

Одна ячейка со списком ID мешает проверять ссылки и выполнять обычные соединения таблиц. В accident_cars каждая строка хранит одну пару (accidentId, carId). Так один автомобиль участвует в разных ДТП, а в одном ДТП может быть несколько автомобилей.

23. Чем отличаются идентифицирующие и неидентифицирующие связи?

При идентифицирующей связи внешний ключ входит в первичный ключ дочерней таблицы. Так устроена accident_cars; в Workbench связь обычно сплошная. В cars.ownerId и полях связей sales внешний ключ не входит в PK: связи неидентифицирующие, обычно пунктирные.

24. Зачем нужна нормализация?

Чтобы уменьшить избыточность и избежать аномалий вставки, изменения и удаления. Например, если имя владельца копировать в каждую строку автомобиля, изменение имени потребует править много строк. В моей модели сведения о гражданине находятся в citizens, а автомобили хранят ссылку ownerId.

25. Что такое первая нормальная форма?

В рамках этой работы поля содержат отдельные значения, а не повторяющиеся группы или списки объектов. Например, участники ДТП представлены отдельными строками accident_cars. Понятие атомарности зависит от задач: строковый адрес не нарушает 1NF автоматически, но его разделение удобно для поиска по частям.

26. Что такое вторая нормальная форма?

Таблица находится в 1NF, а каждый неключевой атрибут зависит от всего кандидатного ключа, а не от его части. Например, если в accident_cars добавить марку автомобиля, она будет зависеть только от carId — это частичная зависимость. Поэтому марка хранится в cars.

27. Что такое третья нормальная форма? Как объяснить ее на своих таблицах?

Практически: таблица находится в 2NF и не содержит транзитивных зависимостей неключевых атрибутов от ключа через другие неключевые атрибуты. Я не храню имя владельца в cars: carId определяет ownerId, а сведения о владельце вынесены в citizens. Формально 3NF оценивается по функциональным зависимостям, а не просто по наличию ID и внешних ключей.

28. Зачем адрес ДТП разделен на city, street и house?

Чтобы отдельно фильтровать и обрабатывать город, улицу и дом. house имеет строковый тип, поскольку возможны значения с буквами и корпусами. Это проектное решение; само разделение адреса еще не доказывает соответствие всей модели 3NF.

29. Почему первичный ключ гражданина — citizenId, а не паспорт?

Это стабильный технический идентификатор, удобный для ссылок. Паспорт может заменяться, поэтому он хранится отдельно с ограничением UNIQUE. Телефон и паспорт не являются числовыми величинами, над которыми нужно выполнять вычисления.

30. Зачем одновременно cars.ownerId и таблица sales?

ownerId хранит текущее состояние — владельца автомобиля сейчас. sales хранит историю событий — кто, кому и когда продал автомобиль. Эти данные нужно согласовывать: последняя продажа и текущий владелец не должны противоречить принятой модели учета.

31. Меняется ли ownerId автоматически после INSERT в sales?

Нет, внешний ключ проверяет существование ссылок, но не обновляет связанные данные. В нашей работе триггера нет. При продаже нужно в одной транзакции добавить запись sales и изменить cars.ownerId, предварительно проверив, что продавец — текущий владелец.

ЛАБОРАТОРНАЯ № 4. ФИЗИЧЕСКАЯ РЕАЛИЗАЦИЯ И ДАМП

32. Назовите этапы проектирования базы. [Самопроверка методички]

Системный анализ предметной области → инфологическое проектирование → выбор СУБД → даталогическое проектирование → физическое проектирование. В нашей работе после проектирования следуют создание объектов на сервере, загрузка записей и проверка результата.

33. Что такое физическая и логическая независимость данных? [Самопроверка методички]

Физическая независимость — возможность менять организацию хранения, например индексы и размещение данных, без изменения логической схемы и пользовательских представлений. Логическая независимость — возможность изменять концептуальную схему, сохраняя внешние представления и приложения через подходящие отображения. Она не означает, что любое удаление столбца проходит для приложения незаметно.

34. В чем суть инфологического проектирования? [Самопроверка методички]

Понять, какие объекты и факты нужно хранить, какие у них свойства и связи, какие действуют правила предметной области. Например, автомобиль имеет текущего владельца, а в одном ДТП могут участвовать несколько автомобилей. На этом этапе важен смысл, а не SQL-синтаксис.

35. Какие типы связей предусмотрены между таблицами? [Самопроверка методички]

1:1, 1:N и M:N. В «ГИБДД» гражданин и его автомобили связаны как 1:N, автомобили и ДТП концептуально — M:N. Последняя связь реализована через accident_cars двумя связями 1:N.

36. Что такое первичный и вторичный ключ? [Самопроверка методички]

Первичный ключ выбран для однозначного определения строки. Термин «вторичный ключ» обычно обозначает дополнительный ключ поиска, который может быть неуникальным; например, фамилию можно индексировать для поиска граждан. Это не внешний ключ; уникальный кандидатный ключ, не выбранный первичным, точнее называется альтернативным. При необходимости стоит уточнить, в каком смысле преподаватель использует термин «вторичный».

37. Как образуется связь между таблицами? [Самопроверка методички]

В дочерней таблице создается FOREIGN KEY, ссылающийся на подходящий ключ родительской таблицы. Например, cars.ownerId ссылается на citizens.citizenId. Типы столбцов должны быть совместимыми; для INT важна также одинаковая знаковость. Простого совпадения названий столбцов недостаточно.

38. Что означает каскадное обновление? [Самопроверка методички]

ON UPDATE CASCADE автоматически меняет внешние ключи дочерних строк при изменении значения родительского ключа. Это не каскадное обновление любых полей. В вашей схеме задано NO ACTION; для InnoDB оно фактически действует как RESTRICT: изменение или удаление родительской записи, нарушающее существующие ссылки, отклоняется.

39. Какие типы данных подходят для текста, чисел, дат, логических значений и больших объектов? [Самопроверка методички]

Текст — CHAR, VARCHAR, TEXT; целые числа — INT и другие целочисленные типы; точные десятичные числа — DECIMAL; приблизительные — FLOAT и DOUBLE; даты и время — DATE, TIME, DATETIME, TIMESTAMP, YEAR. BOOL/BOOLEAN в MySQL являются синонимами TINYINT(1): без CHECK они не ограничивают значения только нулем и единицей. Для бинарного содержимого подходит BLOB; для больших текстов — TEXT. Большие файлы также можно хранить вне базы, сохраняя в ней путь или идентификатор.

40. Что именно выполнено в работе № 4?

Создана физическая база gibdd с пятью таблицами, добавлены десять граждан, десять автомобилей, пять ДТП, десять записей об участии и пять продаж — всего 40 строк. Выполнены SELECT, создан дамп структуры и данных. Связей по внешним ключам шесть, дополнительных CHECK-ограничений пять.

41. Чем Forward Engineering отличается от Reverse Engineering?

Forward Engineering идет от модели к SQL и объектам сервера. Reverse Engineering идет от существующих таблиц сервера или CREATE-скрипта к модели. Поэтому после восстановления дампа я могу получить диаграмму командой Database → Reverse Engineer.

42. Чем различаются .mwb, SQL-скрипт и дамп?

.mwb — файл модели Workbench: таблицы, связи и расположение объектов диаграммы; он не является резервной копией записей на сервере. SQL-скрипт — текст команд; его содержание зависит от задачи. Дамп — SQL-экспорт, который в нашем случае включает структуру и данные, но не сохраняет расположение таблиц на диаграмме.

43. Что проверяют NOT NULL, UNIQUE, CHECK и FOREIGN KEY? Почему нельзя оставить только CHECK?

NOT NULL требует значение, UNIQUE запрещает повтор значений ключа, CHECK проверяет выражение над данными, FOREIGN KEY — существование связанной строки. Например, CHECK (power > 0) не заменяет NOT NULL: результат UNKNOWN из-за NULL сам по себе не приводит к нарушению CHECK. В MySQL CHECK исполняется начиная с версии 8.0.16.

44. Какие CHECK заданы в «ГИБДД»?

power > 0; maxSpeed > 0; price >= 0; releaseYear BETWEEN 1901 AND 2155; sellerId <> buyerId. Последний запрет не доказывает, что продавец действительно текущий владелец: для этого нужна отдельная проверка. Марки автомобилей не ограничены перечнем из десяти тестовых строк — это открытый список предметной области.

45. Почему появилась ошибка 1062? Как связана с ней AUTO_INCREMENT?

Это попытка добавить повтор уникального или первичного ключа. После импорта дампа записи уже были в базе, поэтому повторный INSERT с citizenId = 1 был отклонен. AUTO_INCREMENT не отменяет явно указанный повторяющийся ID; нужно проверить существующие данные, а не отключать ограничения.

46. Что означает ошибка 1452 и почему она возникает при заполнении?

INSERT или UPDATE ссылается через внешний ключ на несуществующую родительскую строку. Например, у cars указан ownerId, которого нет в citizens. Нужно исправить ссылку либо сначала добавить родительскую запись; отключение FOREIGN_KEY_CHECKS не исправляет данные.

47. Для чего нужна транзакция? Можно ли откатить CREATE TABLE?

Транзакция объединяет связанные изменения: START TRANSACTION, затем команды, затем COMMIT либо ROLLBACK. Например, запись продажи и изменение владельца должны примениться вместе. В InnoDB обычные INSERT/UPDATE/DELETE можно откатить до COMMIT; многие DDL-команды MySQL, включая обычные CREATE TABLE, ALTER TABLE и TRUNCATE, вызывают неявную фиксацию и не откатываются как DML. Ошибка одного INSERT сама по себе не гарантирует откат всех предыдущих команд транзакции.

48. Как очистить базу для повторного заполнения, сохранив таблицы?

После резервного копирования удалить дочерние строки раньше родительских: accident_cars → sales → cars → accidents → citizens. Можно выполнить DELETE в одной транзакции и затем снова загрузить подготовленный набор. TRUNCATE имеет другие свойства: неявную фиксацию, сброс автоинкремента и ограничения при наличии внешних ссылок; это не равноценная замена DELETE.

49. Как сохранить отдельную резервную копию «ГИБДД»?

Server → Data Export: выбрать только gibdd и все пять таблиц, Dump Structure and Data, Export to Self-Contained File, Include Create Schema. Нажать Start Export и дождаться Export Completed. В моей работе файл называется gibdd_lab4_dump.sql. Копию нужно хранить отдельно от рабочего сервера.

50. Как восстановить дамп и проверить, что восстановление прошло правильно?

Открыть SQL-дамп в подключении к нужному серверу и выполнить либо использовать Server → Data Import. До запуска проверить, содержит ли дамп DROP TABLE и какие существующие объекты он заменит. После импорта обновить Schemas, проверить таблицы, количество строк, связи и несколько значений. Само наличие файла .sql еще не подтверждает пригодность резервной копии.

51. Как получить сведения из нескольких таблиц одним запросом?

Через JOIN по соответствующим ключам. Для списка автомобилей с фамилиями владельцев:

```
SELECT c.carId, c.brand, p.surname, p.name
FROM gibdd.cars AS c
JOIN gibdd.citizens AS p
  ON p.citizenId = c.ownerId;
```

Для выдач фильмов соединяются issued_films, films и clients по filmId и clientId. JOIN читает данные и сам по себе не создает ограничений FOREIGN KEY.

52. Что еще могут попросить показать непосредственно в Workbench?

Наличие схем: SHOW DATABASES;

Таблицы базы: SHOW TABLES FROM gibdd;

Поля: DESCRIBE gibdd.cars;

Полное определение с ключами и ограничениями: SHOW CREATE TABLE gibdd.cars;

Данные: SELECT * FROM gibdd.citizens;

Количество строк: SELECT COUNT(*) FROM gibdd.citizens;

COUNT(*) считает строки, а COUNT(phone) — только строки с ненулевым phone. Для выборки конкретного гражданина используется WHERE citizenId = 1. Порядок выдачи гарантируется только при явном ORDER BY.

КОРОТКИЙ РАССКАЗ ДЛЯ НАЧАЛА ЗАЩИТЫ

ЛР № 2: «Я реализовал базу проката фильмов. films и clients — справочники, issued_films — журнал выдач с внешними ключами. Составной первичный ключ из фильма, клиента и даты допускает повторную выдачу в разные дни. Модель перенесена на сервер и заполнена учебными записями».

ЛР № 3: «Я спроектировал базу ГИБДД: граждане, автомобили, ДТП, участие автомобилей в ДТП и продажи. В модели определены ключи и шесть внешних связей. Связь автомобилей с ДТП многие-ко-многим реализована через accident_cars. Данные о гражданах и автомобилях разделены, модель нормализована с учетом функциональных зависимостей».

ЛР № 4: «Я физически реализовал ГИБДД на MySQL, заполнил пять таблиц 40 записями, проверил результаты SELECT и экспортировал структуру вместе с данными. Согласованность обеспечивают ключи и CHECK-ограничения. Модель хранится в .mwb, резервная копия записей — в SQL-дампе».

МЕСТА В ОТЧЕТАХ, К КОТОРЫМ СТОИТ ПОДГОТОВИТЬСЯ

1. В PDF лабораторной № 2 есть текстовые INSERT для clients и issued_films, но нет отдельных снимков заполненных этих двух таблиц. Перед защитой полезно добавить такие снимки: SELECT * FROM films.clients; SELECT * FROM films.issued_films;. Листинг доказывает наличие кода, но сам по себе не доказывает выполнение.

2. Записи issued_films ссылаются на конкретные ID 1–5, хотя справочники заполнялись без явных ID. Это работает при фактическом наличии этих ключей; после повторных загрузок ID могут измениться. Умейте показать таблицы films и clients.

3. rentalPrice хранится в записи выдачи как ставка на момент этой операции. Если по вашим требованиям она определяется только фильмом и никогда не меняется между выдачами, возникает зависимость от части составного ключа; тогда постоянную ставку нужно хранить в films или отдельном тарифе. Не утверждайте 3NF без уточнения зависимости.

4. В 3NF не достаточно просто разделить адрес или добавить технические ID. Объясняйте, какие данные от какого ключа зависят.

5. CHECK (sellerId <> buyerId) не обеспечивает принадлежность машины продавцу и не меняет ownerId. Для реальной продажи необходимы проверка текущего владельца и согласованная транзакция; триггер в этих работах не реализован.

6. Модель разрешает одному гражданину иметь несколько автомобилей, даже если в тестовом наборе сейчас каждому назначен один. Кардинальность задается структурой и ограничениями, а не количеством демонстрационных записей.

7. Итоговый отчет № 4 сохранен с исходным титульником № 3 по вашей просьбе. Если преподаватель проверяет номер и тему титульного листа, их стоит согласовать перед сдачей. В исходных снимках также обрезаны заголовки citizens и низ таблицы sales на диаграмме.

8. Не утверждайте, что лично проверили восстановление дампа или отрицательные тесты, если на своем сервере этого не делали. Проверки подготовленных файлов на отдельном сервере выполнялись при подготовке материалов; это отдельный факт от действий в вашем Workbench.

ПОРЯДОК ПОВТОРЕНИЯ

Сначала выучите различия трех работ, свои таблицы и ключи. Затем разберите 1NF–3NF, Forward/Reverse Engineering и различия .mwb/дампа. В конце отработайте в Workbench SELECT, SHOW CREATE TABLE и объяснение ошибок 1062/1452. На защите лучше объяснить ограничение модели и его возможное улучшение, чем утверждать, что схема решает любые случаи.